Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 3

Implementation of Convolutional Neural Networks (CNNs) for Image Classification

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

To understand the working principle of Convolutional Neural Networks by implementing convolution, pooling, feature map visualization, and image classification using TensorFlow/Keras.

2. Learning Outcomes

Students will be able to

1. Explain the convolution operation.
2. Compute output dimensions using stride and padding.
3. Visualize feature maps.
4. Implement max pooling and average pooling.
5. Calculate CNN parameters.
6. Build a CNN for image classification.
7. Evaluate CNN performance.

3. Background Theory

A Convolutional Neural Network consists of

- Convolution Layer
- Activation Layer
- Pooling Layer
- Fully Connected Layer

The convolution operation is

$$Y(i,j) = \sum_m \sum_n X(i+m, j+n) K(m,n)$$

where

- X : Input Image
- K : Kernel (Filter)
- Y : Feature Map

The output feature map size is

$$\frac{N - F + 2P}{S} + 1$$

where

N	Input Size
F	Filter Size
P	Padding
S	Stride

3.1 Common Convolution Kernels

A **kernel** (or **filter**) is a small matrix that slides over an input image to extract important visual features such as edges, textures, corners, and smooth regions. During convolution, the kernel is multiplied element-wise with the corresponding image pixels, and the results are summed to produce a single value in the output feature map. Different kernels highlight different image characteristics.

1. Identity Kernel

The identity kernel preserves the original image without modifying its pixel values.

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

Purpose:

- Preserves the original image.
- Used for understanding the convolution operation.

2. Sobel-X Kernel (Vertical Edge Detection)

The Sobel-X kernel detects **vertical edges** by measuring intensity changes along the horizontal direction.

$$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

Applications:

- Object detection
- Lane detection
- Face recognition

3. Sobel-Y Kernel (Horizontal Edge Detection)

The Sobel-Y kernel detects **horizontal edges** by measuring intensity changes along the vertical direction.

$$\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

Applications:

- Text detection
- Boundary extraction
- Medical image analysis

4. Laplacian Kernel

The Laplacian kernel detects edges in all directions using the second-order derivative of image intensity.

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Applications:
- Edge enhancement
- Satellite image analysis
- Medical imaging

5. Sharpening Kernel

This kernel enhances fine details by emphasizing high-frequency components.

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Applications:
- Image enhancement
- Optical Character Recognition (OCR)
- Face recognition

6. Box Blur Kernel

The Box Blur kernel smooths an image by replacing each pixel with the average of its neighboring pixels.

$$\frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Applications:
- Noise removal
- Image smoothing
- Image preprocessing

7. Gaussian Blur Kernel

Gaussian blur smooths the image while preserving the overall structure better than a simple average filter.

$$\frac{1}{16}\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

Applications:
- Noise reduction
- Computer vision preprocessing
- Feature extraction

8. Emboss Kernel

The Emboss kernel creates a three-dimensional appearance by emphasizing directional changes in intensity.

$$\begin{bmatrix} -2 & -1 & 0 \\ -1 & 1 & 1 \\ 0 & 1 & 2 \end{bmatrix}$$

Applications:
- Texture analysis
- Artistic image effects

9. Outline Detection Kernel

This kernel highlights object boundaries by emphasizing regions with rapid intensity changes.

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

Applications:
- Image segmentation
- Boundary detection
- Shape extraction

10. Motion Blur Kernel

This kernel simulates motion blur along the diagonal direction.

$$\frac{1}{5}\begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

Applications:
- Motion simulation
- Image restoration
- Video processing

Summary of Common Kernels

Kernel	**Size**	**Purpose**	**Applications**
Identity	3×3	Preserve original image	Baseline comparison
Sobel-X	3×3	Detect vertical edges	Object detection
Sobel-Y	3×3	Detect horizontal edges	Boundary detection
Laplacian	3×3	Detect edges in all directions	Medical imaging
Sharpen	3×3	Enhance details	Image enhancement
Box Blur	3×3	Smooth image	Noise removal
Gaussian Blur	3×3	Reduce noise	Computer vision
Emboss	3×3	3D effect	Artistic filtering
Outline	3×3	Boundary extraction	Segmentation
Motion Blur	5×5	Simulate motion	Video processing

4. Numerical Example 1: Convolution

Consider the following input image.

$$X = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

Kernel

$$K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Output
First position

$$1(1) + 2(0) + 4(0) + 5(1) = 6$$

Second position

$$2(1) + 3(0) + 5(0) + 6(1) = 8$$

Third position

$$4(1) + 5(0) + 7(0) + 8(1) = 12$$

Fourth position

$$5(1) + 6(0) + 8(0) + 9(1) = 14$$

Feature Map

$$\begin{bmatrix} 6 & 8 \\ 12 & 14 \end{bmatrix}$$

5. Numerical Example 2: Max Pooling

Feature map

$$\begin{bmatrix} 1 & 5 & 2 & 3 \\ 7 & 8 & 1 & 0 \\ 4 & 6 & 9 & 5 \\ 2 & 3 & 1 & 8 \end{bmatrix}$$

Using a 2×2 max pooling filter,
Window 1

$$\begin{bmatrix} 1 & 5 \\ 7 & 8 \end{bmatrix} \rightarrow 8$$

Window 2

$$\begin{bmatrix} 2 & 3 \\ 1 & 0 \end{bmatrix} \rightarrow 3$$

Window 3

$$\begin{bmatrix} 4 & 6 \\ 2 & 3 \end{bmatrix} \rightarrow 6$$

Window 4

$$\begin{bmatrix} 9 & 5 \\ 1 & 8 \end{bmatrix} \rightarrow 9$$

Output

$$\begin{bmatrix} 8 & 3 \\ 6 & 9 \end{bmatrix}$$

6. Numerical Example 3: Parameter Calculation

Suppose
Input Image

$$32 \times 32 \times 3$$

Convolution Layer

- Filters = 16
- Kernel = 3×3

Parameters

$$(3 \times 3 \times 3 + 1) \times 16$$

$$= (27 + 1) \times 16$$

$= 448$

Therefore,
Total trainable parameters = 448.

7. Dataset

Dataset: CIFAR-10

- Training Images : 50,000
- Testing Images : 10,000
- Classes : 10
- Image Size : $32 \times 32 \times 3$

8. Experimental Procedure

Task 1

Load CIFAR-10.

- Display ten sample images.
- Print dataset dimensions.
- Plot class distribution.

Output:

```
Training data shape: (50000, 32, 32, 3)
Training labels shape: (50000, 1)
Testing data shape: (10000, 32, 32, 3)
Testing labels shape: (10000, 1)
```

Figure 1: Ten sample images from the CIFAR-10 training set with their class labels.

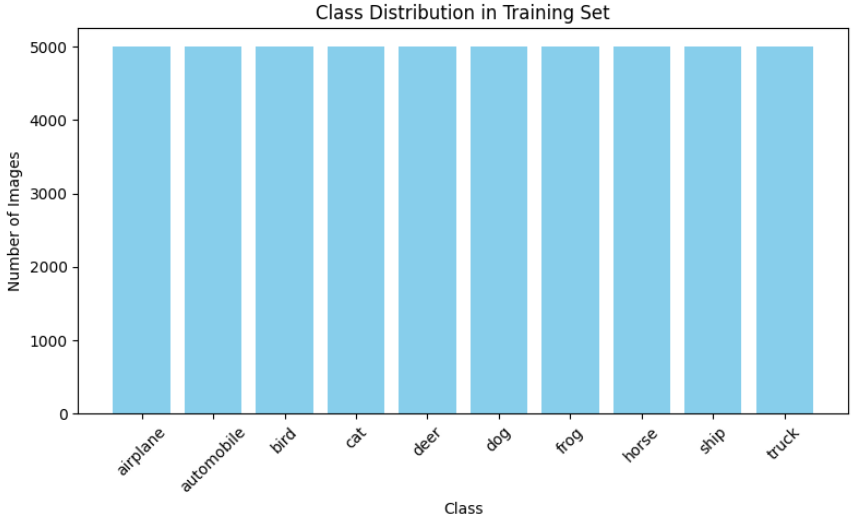

Figure 2: Class distribution of the CIFAR-10 training set.

Inference: The dataset contains 50,000 training images and 10,000 testing images, each of size $32 \times 32 \times 3$, across 10 classes. The class distribution plot shows that all 10 classes are equally represented with 5,000 images each, confirming that CIFAR-10 is a balanced dataset.

Task 2

Implement a convolution layer.

Compare

- Kernel $= 3 \times 3$
- Kernel $= 5 \times 5$
- Kernel $= 7 \times 7$

Record feature map sizes.

Output:

```
Kernel 3x3 -> Feature map shape: (1, 30, 30, 8)
Kernel 5x5 -> Feature map shape: (1, 28, 28, 8)
Kernel 7x7 -> Feature map shape: (1, 26, 26, 8)

Manual calculation (padding='valid', stride=1):
Kernel 3x3 -> Output size: 30x30
Kernel 5x5 -> Output size: 28x28
Kernel 7x7 -> Output size: 26x26
```

Kernel Size	Feature Map Shape	Manual Output Size
3×3	$(1, 30, 30, 8)$	30×30
5×5	$(1, 28, 28, 8)$	28×28
7×7	$(1, 26, 26, 8)$	26×26

Inference: As the kernel size increases, the resulting feature map size decreases, since a larger filter has fewer valid positions to slide across the 32×32 input. The values obtained from the model prediction exactly match the manually computed values using $\frac{N - F + 2P}{S} + 1$.

Task 3

Study hyperparameters.
Compare
- Stride = 1
- Stride = 2
- Padding = Same
- Padding = Valid

Compute output dimensions.
Output:

```
Input size: 32x32, Kernel size: 3x3

Stride=1, Padding=valid -> Output shape: (1, 30, 30, 8)
Stride=2, Padding=valid -> Output shape: (1, 15, 15, 8)
Stride=1, Padding=same  -> Output shape: (1, 32, 32, 8)
Stride=2, Padding=same  -> Output shape: (1, 16, 16, 8)

Manual calculations:
Valid padding, stride 1: 30
Valid padding, stride 2: 15
Same padding, stride 1: 32 (output preserved)
Same padding, stride 2: 16 (output halved)
```

Stride	Padding	Output Shape
1	Valid	$1 \times 30 \times 30 \times 8$
2	Valid	$1 \times 15 \times 15 \times 8$
1	Same	$1 \times 32 \times 32 \times 8$
2	Same	$1 \times 16 \times 16 \times 8$

Inference: A stride of 2 halves the spatial dimensions of the output compared to a stride of 1. "Valid" padding shrinks the feature map since no padding is added around the border, whereas "Same" padding preserves the input size for stride 1 by padding the borders with zeros.

Task 4
Visualize feature maps after the first convolution layer.
Display at least eight feature maps.
Output:

```
Feature maps shape: (1, 32, 32, 32)
```

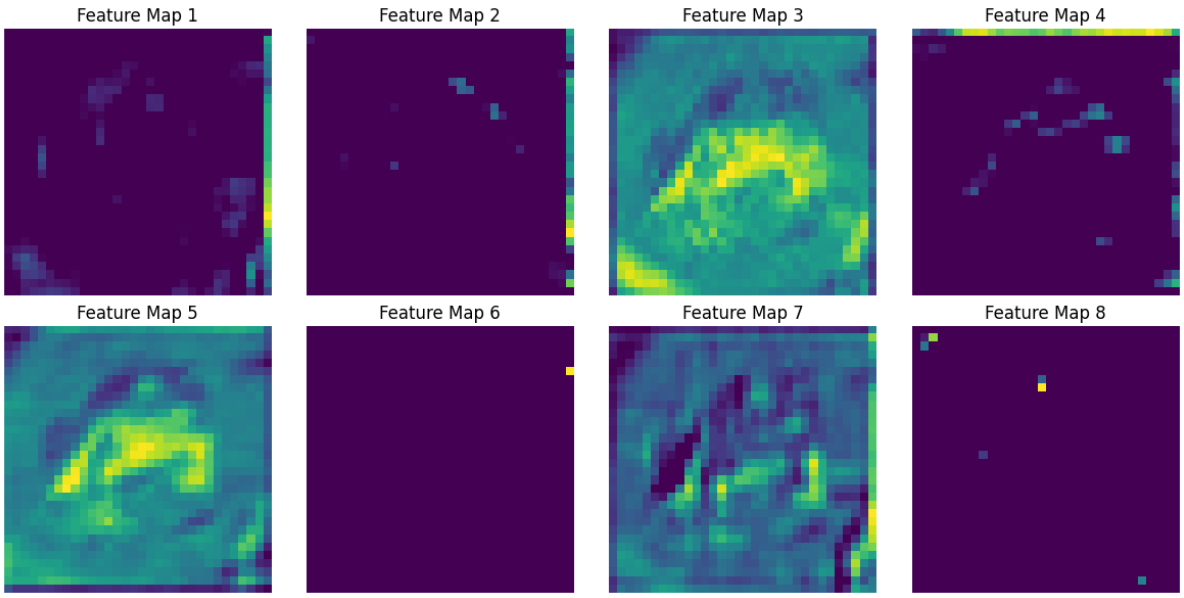

Figure 3: First eight feature maps produced by the first convolution layer (32 filters, 3×3 kernel, ReLU activation).

Inference: The first convolution layer produces 32 feature maps, each of size 32×32 due to "same" padding. Each feature map highlights a different pattern in the input image, such as edges, colour blobs, or textures, showing that different filters learn to detect different low-level visual features.

Task 5
Compare
- Max Pooling
- Average Pooling

Compare output size and accuracy.
Model Architecture (same for both variants):

Layer	**Output Shape**	**Parameters**
Conv2D (32 filters, 3×3)	$(32, 32, 32)$	896
Pooling (Max / Average, 2×2)	$(16, 16, 32)$	0
Conv2D (64 filters, 3×3)	$(16, 16, 64)$	18,496
Pooling (Max / Average, 2×2)	$(8, 8, 64)$	0
Flatten	(4096)	0
Dense	(64)	262,208
Dense (Softmax)	(10)	650
Total Parameters		**282,250**

Output shape after the first pooling layer = $(1, 16, 16, 32)$ for both Max Pooling and Average Pooling.
Training (5 epochs) – Max Pooling:

Epoch	Train Acc.	Train Loss	Val. Acc.	Val. Loss
1	0.4738	1.4743	0.5690	1.2387
2	0.6216	1.0794	0.6478	1.0106
3	0.6665	0.9530	0.6532	0.9948
4	0.6961	0.8758	0.6828	0.9110
5	0.7199	0.8056	0.6868	0.9235

Training (5 epochs) – Average Pooling:

Epoch	Train Acc.	Train Loss	Val. Acc.	Val. Loss
1	0.4522	1.5295	0.5446	1.2642
2	0.5818	1.1888	0.6240	1.0762
3	0.6285	1.0575	0.6272	1.0489
4	0.6617	0.9700	0.6576	1.0047
5	0.6810	0.9069	0.6918	0.9127

Test Set Comparison:

Pooling Type	Test Loss	Test Accuracy
Max Pooling	0.9378	0.6715
Average Pooling	0.9537	0.6684

Inference: Both pooling strategies reduce the spatial dimensions from 32×32 to 16×16 in an identical manner and introduce no additional trainable parameters. Max Pooling converges slightly faster and achieves marginally higher training and test accuracy than Average Pooling, since it retains the strongest (most discriminative) activation in each window, while Average Pooling smooths out feature responses.

Task 6
Construct the following CNN.

$$Input \rightarrow Conv \rightarrow ReLU \rightarrow MaxPool \rightarrow Conv \rightarrow ReLU \rightarrow MaxPool \rightarrow Flatten \rightarrow Dense \rightarrow Softmax$$

Train for
- Optimizer : Adam
- Epochs : 20
- Batch Size : 32

Model Summary:

Layer	Output Shape	Parameters
Conv2D (32 filters, 3×3)	$(32, 32, 32)$	896
MaxPooling2D (2×2)	$(16, 16, 32)$	0
Conv2D (64 filters, 3×3)	$(16, 16, 64)$	18,496
MaxPooling2D (2×2)	$(8, 8, 64)$	0
Flatten	(4096)	0
Dense (ReLU)	(128)	524,416
Dense (Softmax)	(10)	1,290
Total Parameters		**545,098**

Training Log (20 epochs):

Epoch	Train Acc.	Train Loss	Val. Acc.	Val. Loss
1	0.4976	1.3983	0.5766	1.1920
2	0.6420	1.0206	0.6642	0.9653
3	0.6899	0.8818	0.6766	0.9371
4	0.7270	0.7778	0.6518	1.0070
5	0.7554	0.6989	0.7064	0.8798
6	0.7821	0.6180	0.7102	0.8617
7	0.8082	0.5434	0.7152	0.9029
8	0.8318	0.4761	0.7100	0.9197
9	0.8537	0.4108	0.7070	1.0037
10	0.8766	0.3478	0.7060	1.1031
11	0.8958	0.2955	0.7090	1.1560
12	0.9141	0.2452	0.7054	1.2442
13	0.9280	0.2051	0.6992	1.4223
14	0.9359	0.1784	0.6970	1.4858
15	0.9456	0.1531	0.6940	1.6108
16	0.9549	0.1279	0.7102	1.7175
17	0.9594	0.1149	0.6904	1.8619
18	0.9610	0.1101	0.6902	2.0068
19	0.9636	0.1043	0.6982	2.0042
20	0.9668	0.0965	0.6946	2.0502

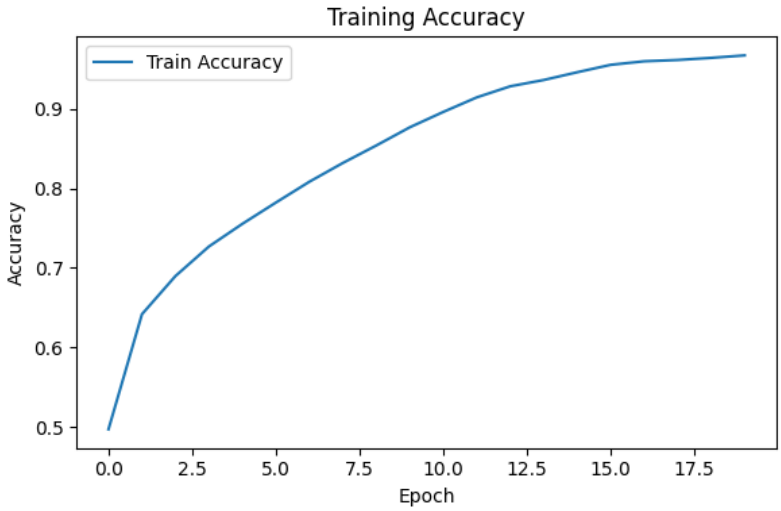

Figure 4: Training accuracy over 20 epochs.

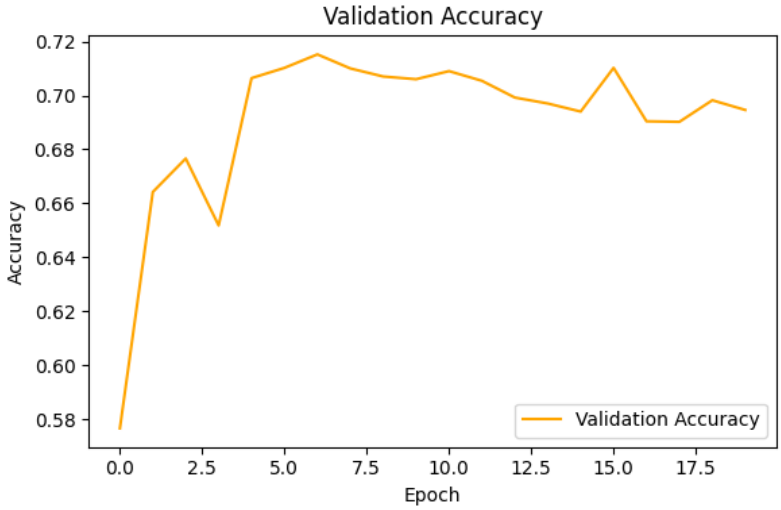

Figure 5: Validation accuracy over 20 epochs.

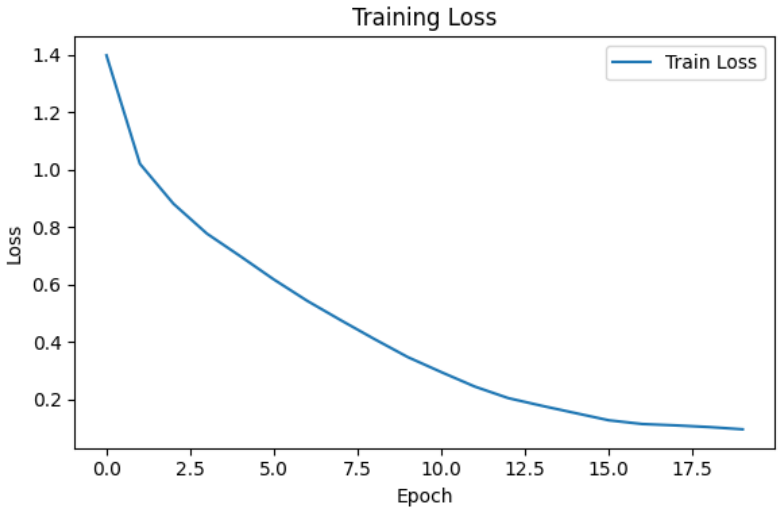

Figure 6: Training loss over 20 epochs.

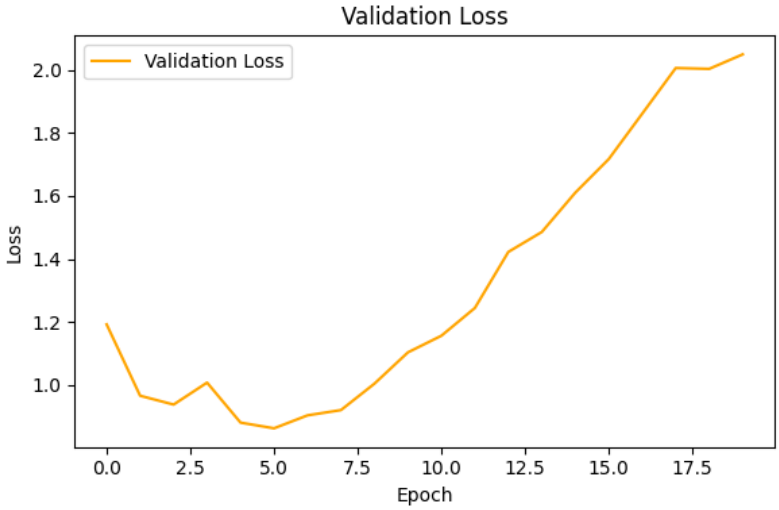

Figure 7: Validation loss over 20 epochs.

Inference: The training accuracy increases steadily and reaches about 96.7% by epoch 20, while the training loss decreases almost monotonically to near 0.10, showing that the model fits the training data very well. However, the validation accuracy plateaus around 69–71% after roughly epoch 6, and the validation loss starts rising after epoch 6, which indicates that the model begins to **overfit** the training data beyond that point.

Task 7

Evaluate

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

Output:

```
Test Accuracy:     0.6785
Precision (macro): 0.6808
Recall (macro):    0.6785
F1-score (macro):  0.6775

Total trainable parameters: 545098
```

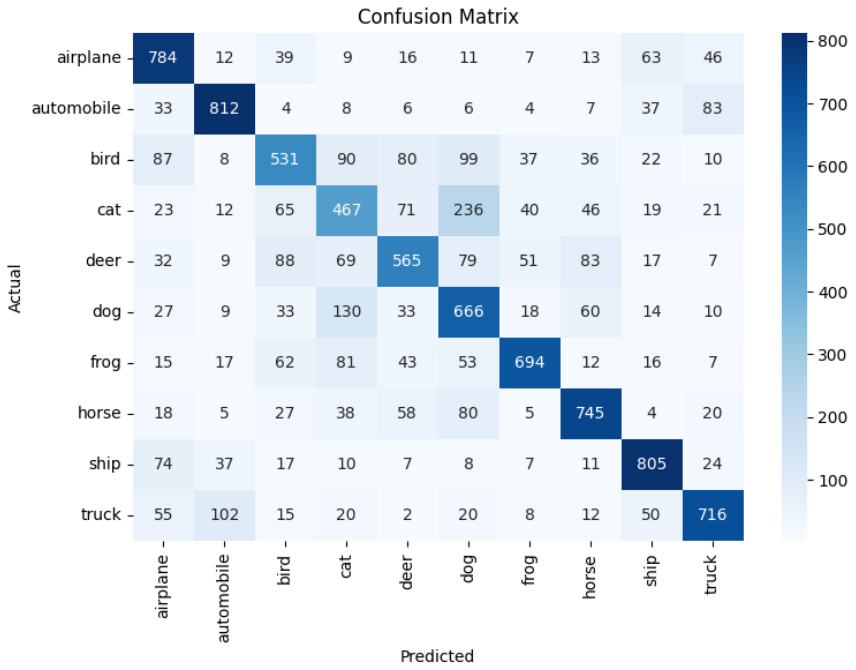

Figure 8: Confusion matrix for the CNN on the CIFAR-10 test set.

Classification Report:

Class	Precision	Recall	F1-score	Support
airplane	0.68	0.78	0.73	1000
automobile	0.79	0.81	0.80	1000
bird	0.60	0.53	0.56	1000
cat	0.51	0.47	0.49	1000
deer	0.64	0.56	0.60	1000
dog	0.53	0.67	0.59	1000
frog	0.80	0.69	0.74	1000
horse	0.73	0.74	0.74	1000
ship	0.77	0.81	0.79	1000
truck	0.76	0.72	0.74	1000
Accuracy		**0.68**		10000
Macro avg	0.68	0.68	0.68	10000
Weighted avg	0.68	0.68	0.68	10000

Inference: The model performs best on visually distinctive classes such as *automobile*, *ship* and *frog*, and struggles most with *cat* and *bird*, which are frequently confused with visually similar animal classes such as *dog* and *deer*. The overall macro-averaged precision, recall and F1-score are all close to 0.68, consistent with the overall test accuracy of 67.85%.

9. Mandatory Plots

Generate

1. Sample Images
2. Class Distribution
3. Training Accuracy
4. Validation Accuracy
5. Training Loss
6. Validation Loss
7. Feature Maps
8. Confusion Matrix

Write a 2–3 line inference for each plot.

10. Additional Exercises

1. Calculate the output size for a 64×64 image using a 5×5 kernel with stride 2 and padding 2.
2. Compute the trainable parameters of a convolution layer having 64 filters of size 3×3 and an RGB input.
3. Compare ReLU and Sigmoid activation functions.
4. Replace Max Pooling with Average Pooling and compare the results.
5. Increase the number of convolution filters from 16 to 64 and analyze the effect on accuracy and computation time.

11. Results

Metric	**Value**
Training Accuracy (final epoch)	96.68%
Testing Accuracy	67.85%
Precision (macro)	0.6808
Recall (macro)	0.6785
F1-score (macro)	0.6775
Number of Trainable Parameters	545,098

The large gap between the training accuracy (96.68%) and the testing accuracy (67.85%) confirms that the network overfits the training data when trained for 20 epochs without regularization (e.g., dropout or data augmentation).

12. Discussion

1. **Why is convolution preferred over fully connected layers for images?**
   Convolution employs small filters that are shared and move across the image, so that each neuron

only examines a local neighbourhood (a local receptive field) rather than the whole image. Because of the sharing of weights the number of parameters is greatly reduced when compared to a fully connected layer, the spatial structure of the image is maintained, and the network becomes translation invariant—meaning a feature identified in one part of the image can also be detected in another.

2. **How does stride affect the feature map size?**
   The size of the stride determines the number of pixels by which the kernel shifts at each step. When the stride is increased, the kernel skips more pixels and as a result produces a smaller output feature map; conversely, a stride of 1 means that the kernel is moved one pixel at a time and this leads to a larger feature map. This is what was found in Task 3, since changing the stride from 1 to 2 with valid padding caused the output to decrease from 30×30 to 15×15.
3. **What is the role of padding?**
   Padding involves adding extra (typically zero-valued) pixels to the border of the input before the convolution operation. When 'valid' padding is used (i.e. no padding), the output feature map shrinks after each convolution and the information at the borders is ignored. 'Same' padding adds just enough pixels to the border so that the output feature map has the same spatial dimensions as the input, which enables the construction of deeper networks without the feature maps disappearing and makes sure that the border pixels are used as frequently as the central pixels.
4. **Why is pooling used?**
   Pooling—whether maximum or average—down-samples the feature maps, which in turn reduces their spatial dimensions. This results in a decrease of the number of parameters and the amount of computation required by the subsequent layers, helps to control overfitting, and gives a certain degree of translation invariance since small changes in the input do not lead to a significant change in the pooled output.
5. **How do feature maps represent image characteristics?**
   A particular filter being applied to the image produces each feature map, so the map shows the areas in which that kind of pattern (for instance, an edge, a corner, a coloured blob, or a texture) occurs. The feature maps in the early layers generally pick up low-level features such as edges and colours, whereas those in the deeper layers combine the earlier features to represent more abstract, high-level characteristics like shapes and parts of objects.
6. **Why do CNNs require fewer parameters than MLPs?**
   In a multi-layer perceptron, each neuron in a given layer is connected to every pixel of the input, which causes the number of weights to increase very rapidly as the size of the image grows. In a CNN, however, a single small kernel (for example, 3×3) is shared and used throughout the whole image, meaning that the number of parameters is determined solely by the size of the kernel and the number of filters, not by the size of the input image. It is this weight sharing that makes CNNs much more parameter-efficient than MLPs when dealing with image data, as shown in Numerical Example 3, where a 3×3 convolution layer with 16 filters applied to a $32 \times 32 \times 3$ image requires only 448 parameters.

13. References

1. Goodfellow et al., *Deep Learning*.
2. Bishop, *Pattern Recognition and Machine Learning*.
3. Haykin, *Neural Networks and Learning Machines*.
4. TensorFlow Documentation.
5. CIFAR-10 Dataset Documentation.